

Event Scraper

Staff-Level Architecture Review & Renovation Plan

Pripravit: Principal Software Engineer / Staff Architect

Datum: 16. april 2026

Verzija dokumenta: 1.0

1. Executive Summary

Event Scraper je funkcionalen MVP ki uspesno pobira ~800 prihodnjih dogodkov iz 39 avtomatiziranih virov in jih razvrsta na 7 medijskih portalov. Sistem ima delujoce parserje za RSS, iCal in vec vrst HTML strani, osnovno deduplikacijo, kategorizacijo in Drupal-kompatibilen JSON izhod. Za enega razvijalca je to soliden proof-of-concept.

Vendar ima sistem **kriticne varnostne ranljivosti** (ponarejen secret_key, privzeto geslo, CSRF, debug mode), **arhitekturne omejitve** (1.500-vrsticni god object, SQLite v produkciji, ni testov, ni deployment modela) in **operativne vrzeli** (ni retry logike, ni audit traila, ni observability) ki preprečujejo produkcijsko rast. 49 rocnih virov je korekten pristop — to je namenski backlog, ne napaka.

Ta dokument predlaga **postopno prenovno brez big-bang migracije**: 30-dnevni plan popravlja kriticne varnostne luknje in refaktorira engine.py, 90-dnevni plan uvede PostgreSQL in background jobs, 6-mesecni plan prinese polno Drupal integracijo, RBAC in observability.

2. Kaj je v tej zasnovi dovolj dobro za MVP

- ✓ YAML-driven source konfiguracija — fleksibilen, razumljiv, razsirljiv model
- ✓ Centralizirano zbiranje + distribucija po portalih — pravilna arhitekturna odlocitev
- ✓ RSS/iCal parserji — zanesljivi, strukturirani viri imajo prednost pred HTML scrapingom
- ✓ 3-nivojska deduplikacija — pravilen pristop (hash + fuzzy + portal check)
- ✓ Drupal JSON format — to_drupal() metoda pravilno mapira na CMS polja
- ✓ Kulturnik.si kot primarni vir — en agregator za ~1300 dogodkov po vseh regijah
- ✓ Health check modul — registracija vseh virov z monitoringom dostopnosti
- ✓ 49 rocnih virov kot evidentirani backlog — to NI napaka, je namenski pristop
- ✓ Completeness score — osnovna metrika kvalitete podatkov
- ✓ event_media workflow statusi — pravilna zasnova za polavtomatsko objavo

3. Ključne produkcijske slabosti

P0 — Kritično (popraviti pred kakršnokoli produkcijsko uporabo):

#	Problem	Tveganje	Datoteka
1	Flask secret_key je placeholder string	Vsakdo lahko ponaredi session cookie in se prijavlja	auth.yaml
2	Privzeto geslo zamenjaj-me-123 verjetno ne obstaja	Nepooblaščen dostop do dashboarda	auth.yaml
3	debug=True hardcoded v app.py	Interaktivni debugger z RCE v produkciji	app.py
4	Auth bypass ko je AUTH_USERS prazen	Prazna konfiguracija = brez avtentikacije	app.py
5	SQLite brez check_same_thread=False	Threading napake pod Flask concurrent requests	models.py
6	to_drupal() odpre svojo DB sejo brez finally	Session leak, SQLite deadlock	models.py

P1 — Visoka prioriteta (popraviti pred prvimi uporabniki):

#	Problem	Tveganje
7	Ni CSRF zaslote na POST endpointih	Cross-site request forgery napadi
8	status parameter ni validiran proti enum	Poljuben string v workflow statusu
9	request.get_json() ni null-checkan	AttributeError na malformed requestih
10	engine.py je 1.519-vrstični god object z 26 metodami	Nemogoče testirati, vzdrževati ali razširiti
11	Ni testov — nobenih	Vsaka sprememba je tvegana
12	Portal ID-ji v published_checker.py ne ujemajo z Portal ID-ji	Published check tiho ne dela nicesar
13	Ni rate limitinga na /login	Brute-force napad na gesla

4. Ciljna arhitektura sistema

Ciljna arhitektura loci sistem v jasne plasti: **ingestion** (pobiranje), **processing** (obdelava), **storage** (hramba), **distribution** (razvrščanje) in **presentation** (API/UI). Vsaka plast je neodvisno testabilna in zamenljiva.

Plast	Odgovornost	Moduli
Ingestion	HTTP fetch, feed parsing, HTML extraction	parsers/, fetcher.py
Normalization	Cisticenje, standardizacija, geocoding	normalizer.py
Enrichment	Kategorizacija, quality scoring	categorizer.py, scorer.py
Deduplication	Hash, fuzzy, cross-source	dedup.py
Storage	PostgreSQL, migracije, ORM	db/models.py, db/migrations/
Distribution	Dodelitev portalom, workflow	distributor.py
Publishing	Drupal push, status tracking	publisher.py
API	REST endpoints, auth, validation	api/
Jobs	Background tasks, scheduler, retries	jobs/
Admin UI	Dashboard, uredniski vmesnik	web/

5. Predlog nove strukture map in modulov

```
event-scraper/  
  config/  
    settings.yaml          # Globalne nastavitve (ne secrets!)  
    sources/               # 88 YAML konfiguracij virov  
  secrets/                 # .gitignore'd – nikoli v VCS  
    .env                  # DB_URL, SECRET_KEY, DRUPAL_API_KEYS  
  src/  
    core/  
      config.py           # Branje YAML + env vars  
      exceptions.py       # Custom exception hierarhija  
      logging.py          # Structured logging setup  
    ingestion/  
      fetcher.py          # HTTP client z retry, timeout, rate limit  
      parsers/  
        __init__.py       # Parser registry (avtomatski discovery)  
        base.py           # BaseParser abstraktni razred  
        rss_parser.py     # RSS/Atom parser  
        ical_parser.py    # iCal parser  
        kulturnik.py      # Kulturnik JSON + RSS  
        mojaobcina.py     # MojaObcina HTML  
        kinodvor.py       # Kinodvor HTML  
        kinosiska.py      # Kino Siska HTML  
        cankarjevdome.py  # CD HTML  
        visitskofjaloka.py # Visit Skofja Loka HTML  
        generic_html.py   # Genericni HTML (iz list_selectors)  
    processing/  
      normalizer.py       # Text cleaning, date normalization  
      categorizer.py      # Event type + audience classification  
      scorer.py           # Quality + completeness scoring  
      dedup.py            # Multi-layer deduplication  
    distribution/  
      assigner.py         # Portal assignment logic  
      published_checker.py # Ze objavljeni na portalih  
    publishing/  
      drupal_client.py    # Drupal REST API client  
      publisher.py        # Workflow state machine  
    db/  
      models.py           # SQLAlchemy modeli  
      migrations/         # Alembic migracije  
      session.py          # Session factory, connection pool  
    api/  
      app.py              # FastAPI/Flask app factory  
      auth.py             # Auth middleware, RBAC  
      routes/  
        events.py         # Event CRUD endpoints  
        editorial.py      # Approve, reject, feature  
        drupal.py         # Drupal integration endpoints  
        monitoring.py     # Health, stats endpoints  
        schemas.py        # Pydantic/Marshmallow validation  
    jobs/  
      scheduler.py        # APScheduler ali Celery beat  
      scrape_job.py       # Periodic scraping  
      health_job.py       # Periodic health check  
      archive_job.py      # Cleanup preteklih dogodkov  
    web/  
      templates/          # Jinja2 templates  
      static/             # CSS, JS  
  tests/  
    unit/  
    integration/  
    fixtures/             # HTML snapshots za parser teste  
    conftest.py  
  alembic.ini
```

```
pyproject.toml
Dockerfile
docker-compose.yml
```

Ključna sprememba: Parserji so posamezni moduli z skupnim vmesnikom (BaseParser). engine.py se razgradi v več modulov. Vsak parser je neodvisno testabilen s HTML fixture datotekami. Registry pattern omogoča dodajanje novih parserjev brez spreminjanja dispatch kode.

6. Predlog podatkovnega modela — PostgreSQL

Zakaj PostgreSQL namesto SQLite: (1) Concurrent write access — SQLite ima en writer naenkrat, Flask/Gunicorn imata vec workerjev. (2) Full-text search za iskanje dogodkov. (3) JSONB za fleksibilna polja. (4) ENUM tipi za workflow statuse. (5) Alembic migracije. (6) Connection pooling. (7) Row-level locking za concurrent editorial workflow.

Tabela	Namen	Ključna polja	Indeksi
events	Centralni model dogodka	id SERIAL PK, title, description TEXT, date_start DATE, date_end DATE, date_published DATE, link VARCHAR(255)	PRIMARY, date_start, date_end, date_published, link
event_versions	Verzioniranje sprememb dogodka	id SERIAL PK, event_id FK, version INT, changed_at TIMESTAMP, user_id FK, changed_by VARCHAR(100), changed_ip VARCHAR(45)	PRIMARY, event_id, version, changed_at, user_id, changed_by, changed_ip
event_media	Povezava dogodek-portal z vsebino	event_id FK + media_id FK = COMPOSITE PK, is_approved BOOLEAN, approved_at TIMESTAMP, is_published BOOLEAN, published_at TIMESTAMP	PRIMARY, event_id, media_id, is_approved, approved_at, is_published, published_at
media_outlets	7 medijskih portalov	id VARCHAR(50) PK, name, url, primary_regions JSONB, secondary_regions JSONB, drupal_api_url VARCHAR(255)	PRIMARY, name, url, primary_regions, secondary_regions, drupal_api_url
sources	88 registriranih virov	id VARCHAR(50) PK, name, base_url, list_url, feed_url, regional_parser_type VARCHAR(50), config JSONB, is_active BOOLEAN	PRIMARY, name, base_url, list_url, feed_url, regional_parser_type, config, is_active
source_runs	Dnevnik vsakega scrape runa	id SERIAL PK, source_id FK, started_at, finished_at TIMESTAMP, is_finished BOOLEAN, error_message TEXT, is_succeeded BOOLEAN	PRIMARY, source_id, started_at, finished_at, is_finished, error_message, is_succeeded
source_health	Tekoci zdravstveni status vira	source_id FK PK, status ENUM(healthy, degraded, broken, unknown), last_check, last_successful_check TIMESTAMP	PRIMARY, source_id, status, last_check, last_successful_check
users	Uporabniki sistema	id SERIAL PK, email VARCHAR(255) UNIQUE, first_name VARCHAR(50), last_name VARCHAR(50), role ENUM(admin, user, moderator)	PRIMARY, email, first_name, last_name, role
audit_log	Revizijska sled vseh akcij	id BIGSERIAL PK, user_id FK, action VARCHAR(50), entity_type VARCHAR(50), entity_id VARCHAR(50), ip VARCHAR(45)	PRIMARY, user_id, action, entity_type, entity_id, ip
publication_attempt	Dnevnik Drupal push poskusov	id SERIAL PK, event_id FK, media_id FK, attempt INT, ip VARCHAR(45), action ENUM(create, update, unpublish)	PRIMARY, event_id, media_id, attempt, ip, action
parser_backlog	Backlog virov brez parserja	id SERIAL PK, source_id FK, url, reason VARCHAR(50), log_status VARCHAR(50), page_title, reason_text TEXT	PRIMARY, source_id, url, log_status, page_title, reason_text

Migracija SQLite → PostgreSQL:

- Uporabi Alembic za schema migracije od začetka
- Zapisi migracijski skript ki prebere SQLite in vstavi v PostgreSQL
- Vzdržuj dual-write obdobje: 1 teden pisanja v obe bazi, nato prerez
- `raw_data` JSONB polje hrani celoten surovi odgovor parserja za debugging

7. Predlog workflow state machine

7.1 Event-Media workflow (event_media.status):

Iz stanja	V stanje	Kdo	Pogoj	Audit
new	approved	editor, admin	Urednik pregleda in odobri	Da
new	skipped	editor, admin	Ni relevantno za portal	Da
approved	queued	system	Drupal client pobere iz queue	Da
approved	new	editor, admin	Urednik prekliče odobritev	Da
queued	pushed	system	Drupal vrne 2xx + nid	Da
queued	approved	system	Drupal vrne napako, retry later	Da
pushed	published	system/drupal	Drupal potrdi objavo	Da
pushed	queued	admin	Ponoven push potreben	Da
published	archived	system	Dogodek je minil	Ne
*	archived	system	date_end < today - 7 dni	Ne

7.2 Neveljavni prehodi (PREPOVEDANO):

Prehod	Razlog
new -> pushed	Preskoci odobritev — urednik MORA pregledati
new -> published	Preskoci celoten workflow
skipped -> approved	Preskozen dogodek potrebuje review, ne re-approve
published -> new	Objavljeni dogodek se ne more vrniti na zacetek
archived -> *	Arhivirani dogodki so zaključeni
* -> approved (brez editor-ja)	Samo clovek lahko odobri

Implementacija: StateMachine razred z dovoljenimi prehodi kot dict. Vsak prehod klice `validate_transition()` ki preveri: (1) ali je prehod dovoljen, (2) ali ima uporabnik pravico, (3) zapise audit log. Ni vec golega `db.execute(update().values(status=X))` brez validacije.

8. Predlog API redesigna

Trenutni API ima nekonsistentne odgovore, manjka validacija, CSRF in auth na nekaterih endpointih. Predlagam prehod na verzioniran REST API z jasno locenim public/internal/admin delom.

Metoda	Endpoint	Namen	Auth	Idempotent
GET	/api/v1/events	Seznam dogodkov s filtri in paginacija	editor+	Da
GET	/api/v1/events/{id}	Podrobni podatki dogodka	editor+	Da
PATCH	/api/v1/events/{id}	Ureditev polj dogodka	editor+	Da (If-Match)
POST	/api/v1/events/{id}/approve	Odobritev za portale	editor+	Da (idempotent key)
POST	/api/v1/events/{id}/skip	Preskok dogodka	editor+	Da
POST	/api/v1/events/batch	Batch operacija (approve/skip)	editor+	Ne
GET	/api/v1/portals	Seznam portalov s statistiko	viewer+	Da
GET	/api/v1/portals/{id}/queue	Odobreni za push	service	Da
POST	/api/v1/portals/{id}/ack	Potrditev uspešne objave	service	Da (nid)
GET	/api/v1/portals/{id}/export	CSV/JSON export	editor+	Da
GET	/api/v1/sources	Seznam virov z zdravjem	viewer+	Da
GET	/api/v1/stats	Statistika sistema	viewer+	Da
GET	/api/v1/audit	Audit log z filtri	admin	Da
POST	/api/v1/auth/login	Prijava	none	Ne
POST	/api/v1/auth/logout	Odjava	any	Da

Primer request/response za POST /api/v1/events/{id}/approve:

Request:

```
POST /api/v1/events/123/approve
Authorization: Bearer {token}
Content-Type: application/json
X-Idempotency-Key: uuid-here
{
  "portal_ids": ["sobotainfo", "pomurec"],
  "featured": true,
  "priority": 2,
  "editor_notes": "Festival - izpostaviti"
}
```

Response 200:

```
{
  "event_id": 123,
  "transitions": [
    {"portal": "sobotainfo", "from": "new", "to": "approved"},
    {"portal": "pomurec", "from": "new", "to": "approved"}
  ]
}
```

Response 409 (conflict):

```
{
  "error": "invalid_transition",
  "message": "Event 123 is already approved for sobotainfo",
  "current_status": "approved"
}
```

9. Predlog izboljšane deduplikacije

Trenutni problem: Fuzzy matching z 'ce katerakoli metrika preseže 85%' je pregrobo. `partial_ratio('Festival Ljubljana', 'Festival Ljubljana presents: Nocturne') = 100%`, kar bo označilo dva različna dogodka kot duplikat.

9.1 Predlagani vec-slojni pipeline:

Sloj	Metoda	Prag	Rezultat
1. Canonical hash	SHA256(canonical_title + date)	100%	Auto-merge
2. Title similarity	Weighted: $0.4 * \text{ratio} + 0.3 * \text{token_sort} + 0.3 * \text{partial_ratio}$	90-92%	Auto-merge
3. Title similarity	Isti weighted score	80-92%	Manual review
4. Organizer match	Ce isti organizator + isti datum + location overlaps $\geq 75\%$ title	90-92%	Auto-merge
5. Cross-source	Ce razlicen vir + isti datum + $\geq 88\%$ title	N/A	Auto-merge
6. Low confidence	Score pod 80%	N/A	Keep separate

9.2 Canonicalization:

- Lowercase + strip diakritike (c->c, s->s, z->z)
- Odstrani locila in odvecne presledke
- Odstrani stop words: 'v', 'na', 'za', 'in', 'z', 'ob', 'pri', 'do'
- Normaliziraj organizatorje: 'SNG Drama Lj.' -> 'sng drama ljubljana'
- Normaliziraj lokacije: 'CD, Gallusova dvorana' -> 'cankarjev dom gallusova dvorana'

9.3 Duplicate confidence score:

Vsak potencialni duplikat dobi **confidence score 0.0-1.0** izracunan kot:

$\text{score} = 0.40 * \text{title_sim} + 0.25 * \text{date_match} + 0.20 * \text{location_sim} + 0.15 * \text{organizer_sim}$

Kjer je `date_match`: 1.0 ce isti datum, 0.5 ce +/-1 dan, 0.0 sicer.

Auto-merge: $\text{score} \geq 0.92$ | Manual review: 0.75-0.92 | Keep separate: < 0.75

10. Predlog izboljšane kategorizacije

Trenutni problem: Golo keyword matching z 'if pattern in search_text' proizvaja false positive-e. 'pop' ujame 'popularen', 'trg' ujame 'treking'. Default 'vsi' za target_audience je semantico napacen — pomeni 'ne vemo', ne 'za vse'.

10.1 Izboljsan sistem:

- **Word boundary matching:** Uporabi regex `\bpattern\b` namesto substring in
- **Weighted patterns:** Vsak vzorec ima utez (0.0-1.0), končni score je sestevek
- **Multi-label:** Dogodek ima lahko vec tipov: ['koncert', 'festival']
- **Confidence score:** 0.0-1.0 za vsako kategorijo, ne samo da/ne
- **Unknown namesto 'vsi':** Če ni ujemanja, nastavi 'unknown' — urednik dopolni
- **Synonym dictionary:** 'jazz' -> koncert (0.9), 'jazz festival' -> festival (0.95)
- **Editor feedback loop:** Ko urednik popravi kategorijo, beleži kot training data
- **Lemmatizacija:** classla ali lemmagen za slovenscino (dolgorocno)

10.2 Primer confidence score:

```
Naslov: "Jazz festival Maribor 2026"
Kategorije iz vira: "Glasba, Festivali"

Matched patterns:
"jazz"      -> koncert (weight=0.7, boundary=True)
"festival"   -> festival (weight=0.9, boundary=True)
"glasba"    -> koncert (weight=0.8, boundary=True, from categories)

Result:
event_type: ["festival", "koncert"]
confidence: {"festival": 0.90, "koncert": 0.75}
primary_type: "festival" (highest confidence)
```

11. Predlog quality scoring modela

Trenutni completeness score steje izpolnjena polja. Potrebujemo vec-dimenzionalen quality score ki uposteva vsebino, vir, in konsistentnost.

11.1 Formula:

```
quality_score = (  
    0.15 * has_title +                # 1.0 ce obstaja, 0.0 sicer  
    0.20 * description_quality +      # min(len(desc)/200, 1.0) – daljsi opis = boljsi  
    0.15 * has_date_and_time +        # 1.0 ce oba, 0.5 ce samo datum, 0.0 sicer  
    0.10 * has_location +            # 1.0 ce obstaja, 0.0 sicer  
    0.10 * has_image +               # 1.0 ce obstaja IN je dosegljiva, 0.0 sicer  
    0.05 * has_organizer +            # 1.0 ce obstaja, 0.0 sicer  
    0.05 * has_price_info +           # 1.0 ce obstaja (tudi "vstop prost"), 0.0 sicer  
    0.10 * has_category +             # 1.0 ce event_type != null/unknown, 0.0 sicer  
    0.10 * source_reliability +       # source_health.reliability_score (0.0-1.0)  
    0.05 * freshness                 # 1.0 ce scraped < 24h, 0.5 < 7d, 0.0 sicer  
)
```

Penalizacije:

- 0.10 ce dedup_confidence med 0.75-0.92 (potencialni duplikat)
- 0.20 ce image_url vrne 404
- 0.05 ce date_start < today (pretekel dogodek ki je se v bazi)

11.2 Source reliability score:

Metrika	Utez	Izracun
Uspesnost scrapanja	0.30	success_runs / total_runs (30 dni)
Povprečno število dogodkov	0.20	min(avg_events / 20, 1.0)
Konsistentnost	0.20	1.0 - (stddev_events / avg_events)
Casovna stabilnost	0.15	dni_od_zadnjega_outage / 30
Podatkovni coverage	0.15	avg_completeness vseh dogodkov iz vira

12. Predlog varnostnega modela

Podrocje	Trenutno stanje	Predlog
Uporabniki	YAML datoteka, en nivo	PostgreSQL users tabela z RBAC (admin/editor/viewer)
Secret key	Placeholder v YAML	Env var FLASK_SECRET_KEY, generiran z secrets.token_hex(32)
Gesla	bcrypt v YAML	bcrypt v bazi, password reset flow z email tokenji
CSRF	Ni zaiscite	Flask-WTF CSRF tokeni na vseh POST formah
Session mgmt	Flask session, ni expiry	Session expiry 8h, secure + httponly cookie flags
Service auth	Ni — Drupal endpointi so odprti	API key ali JWT za Drupal service-to-service
Secrets storage	V YAML datotekah	.env datoteka (gitignored), env vars v produkciji
Rate limiting	Ni	Flask-Limiter: 5 /login/min, 100 req/min/user
Audit	Ni	audit_log tabela z vsako status spremembo, prijavo, odobritvijo
debug mode	Hardcoded True	Env var FLASK_DEBUG=0, nikoli True v produkciji

12.1 RBAC model:

Vloga	Pravice
viewer	Bere dogodke, statistiko, health — NE more odobravati ali urejati
editor	Vse viewer + approve, skip, feature, edit, batch operations
admin	Vse editor + user management, source config, audit log, force status changes
service	Samo Drupal queue/ack endpointi — API key auth, brez session

13. Predlog audit traila

Vsaka uredniška in sistemska akcija mora biti zabeležena v audit_log tabeli. To omogoča: (1) forenzično analizo, (2) undo operacij, (3) compliance, (4) debugging workflowa.

Akcija	Entity	Podatki
user.login	user	email, ip, user_agent, success/fail
user.logout	user	email
user.created	user	email, role, created_by
event.approved	event_media	event_id, media_id, approved_by, featured, priority
event.skipped	event_media	event_id, media_id, skipped_by, reason
event.status_changed	event_media	event_id, media_id, from_status, to_status, changed_by
event.edited	event	event_id, changed_fields (old + new values)
event.featured	event_media	event_id, media_id, featured_by
drupal.pushed	publication_attempts	event_id, media_id, drupal_nid, response_code
drupal.push_failed	publication_attempts	event_id, media_id, error_message
source.config_changed	sources	source_id, changed_fields
scrape.completed	source_runs	source_id, events_found, events_new, duration
system.dedup_merge	events	kept_event_id, merged_event_id, confidence_score

14. Predlog observability in deployment modela

14.1 Containerizacija:

```
# docker-compose.yml
services:
  web:                                # Flask/FastAPI app (Gunicorn, 2 workers)
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [db, redis]

  worker:                             # Background job worker (scrapping, health checks)
    build: .
    command: python -m jobs.worker
    env_file: .env
    depends_on: [db, redis]

  scheduler:                          # Cron scheduler (APScheduler)
    build: .
    command: python -m jobs.scheduler
    env_file: .env

  db:                                 # PostgreSQL 16
    image: postgres:16-alpine
    volumes: ["pgdata:/var/lib/postgresql/data"]

  redis:                              # Job queue + cache
    image: redis:7-alpine
```

14.2 Background jobs:

Job	Interval	Trenutno	Predlog
Scrape all sources	6h	Sinhroni CLI (python -m scraper.py)	Periodic job z retry
Health check	1h	Sinhroni CLI	Background job
Categorize uncategorized	Po vsakem scrapu	Inline scrape	Post-scrape hook
Archive past events	1x/dan	Ni implementirano	Nightly job
Quality score recalc	1x/dan	Inline ob scrpanju	Nightly batch job
Published check	Pred vsakim scrapu	Inline	Locen pre-scrape job
Image URL validation	1x/teden	Ni implementirano	Weekly job
Drupal push queue	5min	Ni implementirano	Periodic job

14.3 Structured logging + Metrics:

- structlog za JSON loge z correlation ID-ji
- Sentry za error tracking (brezplacen za open source)
- Prometheus metrics: scrape_duration, events_per_source, dedup_matches, api_latency
- Healthcheck endpoint /health za load balancer
- Alert: ce source_health.consecutive_errors >= 3 (email ali Slack webhook)

14.4 Backup:

- PostgreSQL: pg_dump daily (7 dni retencije)
- Source YAML configs: git versioned
- Secrets: nikoli v gitu — env vars ali secrets manager

15. Predlog testne strategije

Nivo	St. testov	Kaj testiramo	Orodja
Unit	~80	Canonicalization, date parsing, categorizer, scorer, dedup logic, state machine	pytest
Parser fixtures	~20	Vsak parser z shranjenim HTML/RSS/iCal snapshotom	pytest + fixtures/
Snapshot	~10	HTML extraction ne producira regresij	pytest-snapshot
Contract	~5	Drupal JSON payload shape ostane stabilen	pytest + jsonschema
Integration	~15	API endpoints, DB operations, auth flow	pytest + TestClient
E2E workflow	~5	Celoten flow: scrape -> categorize -> assign -> approve -> export	pytest
Migration	~3	Alembic up + down migracije so reversibilne	pytest + alembic
Performance	~3	Dedup z 10k eventi, categorizer z 5k eventi	pytest-benchmark

Primer parser fixture testa:

```
# tests/fixtures/kinodvor_2026_04.html – shranjen HTML snapshot
# tests/unit/test_kinodvor_parser.py

def test_kinodvor_extract_events():
    html = Path("tests/fixtures/kinodvor_2026_04.html").read_text()
    parser = KinodvorParser()
    events = parser.parse(html, config)

    assert len(events) >= 10
    assert events[0]["title"] is not None
    assert events[0]["date_start"] is not None
    assert events[0]["image_url"].startswith("http")
```

16. 30-dnevni plan

Teden	Naloga	Rezultat
1	P0 varnostne popravke: secret_key, geslo, debug, auth bypass, Shit stare zapravo uporabo	Shit stare zaprabo uporabo
1	Fix to_drupal() session leak — prenesi featured kot parameter	Ni vec session leakov
1	Fix published_checker portal ID-ji — sinhroniziraj z media.yaml	Published check dejansko dela
2	Razbij engine.py: izvleci parserje v locene module (parsers/)	Vsak parser je svoja datoteka
2	Parser registry pattern: config.parser_type -> avtomatski lookup	Dodajanje parserja brez spreminjanja dispatch
2	Dodaj CSRF zaščito (Flask-WTF)	POST endpointi zaščiteni
3	Osnovna testna infrastruktura: pytest setup, 5 parser fixture test	CI-ready test suite
3	Input validacija na vseh API endpointih	Ni vec AttributeError na malformed requestih
3	Status enum validacija v workflow	Nemogoc neveljaven status
4	Dockerfile + docker-compose za lokalni dev	Reproducibilno okolje
4	Alembic setup za migracije (se vedno SQLite)	Schema migracije od zdaj naprej
4	Structured logging z structlog	JSON logi z correlation ID

17. 90-dnevni plan

Mesec	Naloga	Rezultat
2	PostgreSQL migracija z dual-write obdobjem	Robustna baza za concurrent access
2	RBAC: users tabela, admin/editor/viewer vloge	Granularen dostop
2	Audit log tabela + middleware	Revizijska sled vseh akcij
2	Background jobs z RQ ali Celery (scraping, health check)	Asinhrono scraping
3	Izboljsana deduplikacija: canonicalization + weighted scoring + confidence	Manj false positive/negative
3	Izboljsana kategorizacija: word boundary + multi-label + confidence	Manj natančna klasifikacija
3	Quality scoring model	Prioritizacija po kakovosti
3	Source backlog management: prioritizacija, complexity estimate	Sistematicno reševanje rocnihi virov
3	Sentry integracija za error tracking	Proaktivno odkrivanje napak
3	Staging okolje na VPS	Testiranje pred produkcijo

18. 6-mesecni plan

Mesec	Naloga	Rezultat
4	Drupal push integracija (dejanski REST API calls)	Polavtomatska objava
4	Image proxy/cache (namesto hotlinkanja)	Ni vec pravnih/operativnih tveganj s slikami
4	Event versioning (event_versions tabela)	Zgodovina sprememb

Mesec	Naloga	Rezultat
5	FastAPI migracija (postopna, route-by-route)	Async, boljša validacija, OpenAPI docs
5	Prometheus metrics + Grafana dashboard	Operativna vidljivost
5	Avtomatizacija 10-15 ročnih virov (WordPress /feed/, iCal discovery) v avtomatiziranih virov	
6	Geocoding lokacij (Nominatim)	Lokacijska preciznost
6	ML kategorizacija (classla za slovenscino)	Natančna klasifikacija brez keyword matchinga
6	Multi-tenant: priprava za nove medije/portale	Skalabilnost

19. Top 15 prioritet po vplivu in nujnosti

#	Prioriteta	Vpliv	Nujnost	Cas
1	Popravki PO varnostnih ranljivosti (secret_key, djangoauth)	Visok	Takoj	2h
2	Fix to_drupal() session leak	Visok	Takoj	1h
3	Fix published_checker portal ID mismatch	Visok	Takoj	1h
4	Razbij engine.py v locene parserje	Visok	Teden 2	2-3 dni
5	Dodaj CSRF zaščito	Visok	Teden 2	2h
6	Input validacija na API endpointih	Srednji	Teden 3	4h
7	Status enum validacija v workflow	Srednji	Teden 3	2h
8	Osnovna test suite (parser fixtures)	Visok	Teden 3	2 dni
9	Docker setup za lokalni dev	Srednji	Teden 4	1 dan
10	Alembic migracije	Srednji	Teden 4	4h
11	PostgreSQL migracija	Visok	Mesec 2	3-5 dni
12	RBAC + users v bazi	Srednji	Mesec 2	2 dni
13	Audit log	Srednji	Mesec 2	2 dni
14	Izboljšana deduplikacija	Visok	Mesec 3	3-4 dni
15	Background jobs za scraping	Visok	Mesec 2	3 dni

20. Glavna migracijska tveganja in kako jih obvladati

Tveganje	Verjetnost	Vpliv	Mitigacija
Izguba podatkov pri SQLite-SQL migraciji	Visoka	Kritičen	Dual-write 7 dni, verificiraj števila pred prerezom, ohrani SQLite backup 30 dni
Parser regresije po refaktorju v pythonu	Visoka	Visok	Najprej napisi fixture teste za vsak parser, sele nato refaktoriraj
Downtime med migracijami	Nizka	Srednji	Blue-green deploy, migracije v off-peak urah
Deduplikacija pobriše prave stvari	Srednja	Visok	Soft-delete (archived status), ne hard delete. Manual review za 75-92% confidence
Auth spremembe zaklenejo uporabnike	Nizka	Visok	Ohrani YAML fallback 2 tedna po migraciji v bazo
Drupal API integracija ne deluje	Srednja	Srednji	Ohrani CSV export kot fallback za ročni uvoz
Background jobs ne procesirajo	Nizka	Visok	Dead letter queue + alert ce job ne konca v 30min

Dodatek: Konkretni komentarji na 10 identificiranih točk

1. Parserji zmesani v engine.py

Kaj je narobe: 10 site-specific parserjev, 1.519 vrstic, 26 metod v enem razredu. ScraperEngine je god object ki dela fetch, parse, dedup, DB, media assignment in orkestracijo.

Zakaj je tveganje: Nemogoče testirati posamezen parser brez celotnega engine-a. Dodajanje novega parserja zahteva urejanje dispatch if/elif verige. Dva parserja (mgml, kinosiska) se dispatchata po config.id namesto parser_type — nekonsistentno.

Najhitrejša resitev: Izvleci vsak parser v svojo datoteko z BaseParser vmesnikom: parse(html, config) -> list[dict]. Registry dict mapira parser_type na razred. Dispatch postane parser_cls = REGISTRY[config.parser_type]; events = parser_cls().parse(html, config).

Dolgoročna resitev: Autodiscovery parserjev iz parsers/ direktorija. Vsak parser registrira sam sebe z dekoratorjem @register_parser('kulturnik-rss').

Nujnost: Teden 2. Brez tega ni mogoče pisati testov za parserje.

2. SQLite ni dobra baza za produkcijski workflow

Kaj je narobe: SQLite ima en writer naenkrat. Flask z Gunicorn/multiple workers bo produciral locking napake. Ni full-text searcha, ni JSONB, ni ENUM tipov.

Najhitrejša resitev: Dodaj check_same_thread=False na engine in WAL mode. To resi threading problem za kratkoročno.

Dolgoročna resitev: PostgreSQL z Alembic migracijami. Docker compose z PG containerjem za lokalni dev.

Nujnost: check_same_thread takoj (1h). PostgreSQL v mesecu 2.

3. Uporabniki v auth.yaml

Kaj je narobe: Gesla (hash) in secret_key v datoteki ki bo verjetno v git repo. Ni vlog, ni password reseta, ni audit logiranja prijav.

Najhitrejša resitev: Takoj zamenjaj secret_key z naključnim. Premakni v .env datoteko ki je v .gitignore.

Dolgoročna resitev: Users tabela v bazi z RBAC, password reset flow, audit log prijav.

Nujnost: Secret key popravek TAKOJ. Users v bazo v mesecu 2.

4. API statusi in workflow semantika

Kaj je narobe: Status se nastavi z golim db.execute(update().values(status=X)) brez validacije. Klient lahko pošlje status='banana' in se zapiše v bazo. Ni prehodni matrike — vsak status lahko preide v vsakega.

Najhitrejša resitev: Enum validacija: ALLOWED_STATUSES = {...}. V vsakem endpointu preveri da je new_status v ALLOWED.

Dolgoročna resitev: StateMachine razred z validate_transition(from, to, role). PostgreSQL ENUM tip za status stolpec.

Nujnost: Enum validacija teden 3. State machine mesec 2.

5. Deduplikacija pregrobo

Kaj je narobe: 'Ce katerakoli fuzzy metrika preseže 85%' producira false positive. partial_ratio je posebej problematičen — kratki naslovi so podmnožica dolgih. is_duplicate() ima mrtev fuzzy_threshold parameter ki nikoli

ni uporabljen.

Najhitrejsa resitev: Dvigni prag na 90%. Uporabi weighted average namesto max(). Odstrani mrtev parameter.

Dolgorocna resitev: Confidence score z canonicalizacijo (poglavje 9).

Nujnost: Quick fix teden 3. Polna prenova mesec 3.

6. 'vsi' kot default target audience

Kaj je narobe: 'vsi' semanticno pomeni 'ta dogodek je namenjen vsem'. V resnici pomeni 'ne vemo komu je namenjen'. To onemogoča filtriranje.

Resitev: Zamenjaj default z 'unknown'. Ko urednik pregleda, nastavi pravo vrednost. Dogodki z 'unknown' so vizualno označeni v dashboardu.

Nujnost: Teden 3 (enostavna sprememba).

7. Image hotlinkanje

Kaj je narobe: image_url kaže na tuji strežnik. Če lastnik spremeni URL, slika izgine. Nekateri strežniki blokirajo hotlinking (403). Pravno gledano je to potencialna kršitev avtorskih pravic.

Najhitrejsa resitev: Preveri dostopnost slik enkrat tedensko (image validation job). Oznaci nedosegljive slike.

Dolgorocna resitev: Image proxy ki shrani lokalno kopijo, servira iz lastnega CDN. S3-compatible storage (MinIO za self-hosted).

Nujnost: Validacija teden 4. Proxy mesec 4.

8. Manjka audit trail

Kaj je narobe: Nobena uredniška odločitev ni zabeležena. Če urednik po pomoti odobri 500 dogodkov, ni načina ugotoviti kdo, kdaj, zakaj.

Resitev: audit_log tabela + AuditMiddleware ki avtomatsko zapise vsak POST/PATCH/DELETE. Primer v poglavju 13.

Nujnost: Mesec 2 (skupaj z RBAC in PostgreSQL).

9. Manjka testna strategija

Kaj je narobe: 0 testov. Vsaka sprememba v parserju je tvegana — ni načina vedeti ali je refaktor pokvaril ekstrakcijo za obstoječe vire.

Najhitrejsa resitev: pytest setup + 5 parser fixture testov (en HTML snapshot za vsak parser tip). To traja 1 dan in takoj prinese varnostno mrežo.

Dolgorocna resitev: Polna testna piramida iz poglavja 15.

Nujnost: Teden 3. PRED kakršnimkoli večjim refaktorjem.

10. Manjka deployment in observability

Kaj je narobe: Sistem se zaganja z python3 web/app.py. Ni Docker-ja, ni process managerja, ni log agregacije, ni alertov. Če scraper crkne ob 3h zjutraj, nihče ne ve.

Najhitrejsa resitev: Dockerfile + docker-compose (teden 4). systemd unit za scraper cron job na VPS.

Dolgorocna resitev: docker-compose z web + worker + scheduler + db + redis (poglavje 14).

Nujnost: Docker teden 4. Polna observability mesec 3.

Kaj bi popravil v prvem tednu

- Zamenjaj `secret_key` v `auth.yaml` z naključnim 64-char hex stringom
 - Zamenjaj privzeto geslo `zamenjaj-me-123`
 - Spremeni `debug=True` v `os.environ.get('FLASK_DEBUG', '0') == '1'`
 - Dodaj `check_same_thread=False` na SQLite engine
 - Popravi `to_drupal()` — `featured` naj bo parameter, ne odpira svoje session
 - Sinhroniziraj portal ID-je v `published_checker.py` z `media.yaml`
 - Dodaj `.gitignore` vnose za `auth.yaml`, `.env`, `data/events.db`
 - Dodaj explicit auth guard na `auth_required`: ce `AUTH_USERS` prazen, zahtevaj `env var ALLOW_DEV_MODE=1`
-

Kaj bi prepovedal pred produkcijsko rastjo

- Deploy brez CSRF zaščite na POST endpointih
 - Dodajanje novih parserjev v `engine.py` (najprej razbij)
 - Shranjevanje secretov v YAML datotekah ki gredo v git
 - Kakrsenkoli write endpoint brez input validacije
 - Workflow status sprememba brez enum preverjanja
 - Produkcijski deploy brez vsaj 20 osnovnih testov
 - Flask dev server v produkciji (mora biti Gunicorn ali uWSGI)
 - SQLite z vec kot 2 concurrent workerji
-

Kaj bi pustil pri miru v tej fazi

- ✓ YAML konfiguracija virov — deluje, je pregledna, jo razumejo ne-razvijalci
 - ✓ 49 rocnihi virov — so korektno evidentirani backlog, ne tehnicni dolg
 - ✓ Kulturnik RSS kot primarni vir — je zanesljiv, strukturiran, pokrije vso Slovenijo
 - ✓ SQLite za lokalni development (PostgreSQL samo za staging/production)
 - ✓ Flask (ne FastAPI) — migracija je smiselna sele v mesecu 5 ko je vec API endpointov
 - ✓ Keyword matching za kategorizacijo — ML pristop je smislen sele z 5000+ labeliranimi eventi
 - ✓ Geocoding lokacij — lepo imeti, ni kriticno za MVP
 - ✓ Email notifikacije — Slack webhook ali email alert ko `source_health` degradira, ampak ne takoj
 - ✓ Image proxy — hotlinkanje je sprejemljivo za MVP, dolgorocno ne
 - ✓ Multi-tenant podpora — 7 portalov je dovolj za trenutno arhitekturo
-